CSL 7640: Natural Language Understanding - Assignment 2

Word Embeddings and Character-Level Sequence Generation

Name: Chirugudu Noah Sujiv Raj
Roll No: M25CSE011

February 25, 2026

1 Problem 1: Learning Word Embeddings from IIT Jodhpur Data

1.1 Introduction and Problem Statement

This assignment required training two distinct Word2Vec architectures—Continuous Bag of Words (CBOW) and Skip-gram with Negative Sampling—on a corpus of institutional documents from IIT Jodhpur.

1.2 Dataset and Preprocessing

1.2.1 Data Collection

The dataset was collected from multiple sources within the IIT Jodhpur institutional domain, including official website pages, academic regulation documents, course materials, faculty profiles, and administrative circulars. The collection process employed automated web scraping with targeted PDF extraction, capturing a breadth of institutional discourse spanning academic programs, administrative procedures, and research initiatives.

Table 1: Dataset Statistics from Preprocessing Phase

Metric	**Unit**	**Value**
Number of Documents	count	460
Total Tokens	count	698,783
Vocabulary Size (raw)	types	27,873
Vocabulary Size (filtered, min_count=2)	types	19,148
Corpus File Size	MB	5.48

1.2.2 Preprocessing Pipeline

The preprocessing pipeline implemented several critical stages. Raw text was normalized through lowercasing, punctuation removal, and URL stripping. Notably, standard English stopwords were filtered, but domain-specific acronyms (e.g., "ug", "pg", "cse", "btech", "mtech") were preserved as they carry substantial semantic value within the institutional context. The vocabulary was subsequently filtered to retain only tokens appearing at least twice in the corpus, reducing vocabulary size from 27,873 to 19,148 types while eliminating noise from single occurrences.

Table 2: Most Frequent Tokens in Corpus

Rank	**Token**	**Frequency**
1	lectures	10,207
2	engineering	6,804
3	tech	5,487
4	course	5,288
5	learning	4,637
6	design	4,274
7	students	3,918
8	systems	3,706
9	department	3,346
10	data	3,315

The high frequency of technical terms (lectures, course, learning, design, systems) reflects the academic nature of the corpus. Engineering-related vocabulary dominates, which is expected given the institutional context.

1.3 Methodology: Word2Vec Architectures

1.3.1 Continuous Bag of Words (CBOW)

The CBOW model predicts a target word from context words within a sliding window. Mathematically, given context words $\{w_{t-c}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+c}\}$, the model learns

to predict w_t by:

$$P(w_t|\text{context}) = \frac{\exp(\mathbf{v}_{w_t}^T \mathbf{v}_c)}{\sum_{w' \in V} \exp(\mathbf{v}_{w'}^T \mathbf{v}_c)} \tag{1}$$

where $\mathbf{v}_c$ is the average of context word vectors and $\mathbf{v}_{w_t}$ is the target word vector.

1.3.2 Skip-gram with Negative Sampling

Skip-gram inverts the prediction direction: given a target word, it predicts context words. This architecture often produces superior semantic properties. Rather than computing full softmax (computationally expensive), negative sampling contrasts the true context word against k randomly sampled negative examples:

$$\mathcal{L} = \log \sigma(\mathbf{v}_w^T \mathbf{v}_c) + \sum_{i=1}^{k} \log \sigma(-\mathbf{v}_{n_i}^T \mathbf{v}_c) \tag{2}$$

where σ is the sigmoid function, $\mathbf{v}_w$ and $\mathbf{v}_c$ are word and context vectors respectively, and n_i are negative samples.

1.3.3 Experimental Configurations

Multiple configurations were systematically explored:

Table 3: Word2Vec Model Configurations Trained

Config ID	Architecture	Dimension	Window	Neg. Samples
SG1	Skip-gram	50	3	5
SG2	Skip-gram	300	5	5
SG3	Skip-gram	100	5	10
CBOW1	CBOW	50	3	5
CBOW2	CBOW	300	5	5
CBOW3	CBOW	100	5	10

These configurations allow exploration of dimension effects (50, 100, 300), context window sizes (3, 5), and negative sampling strategies (5, 10).

1.4 Training Results

The models were trained for 3 epochs over the preprocessed corpus. Figure 1 presents the training loss trajectory for the SG2 configuration, demonstrating typical learning

dynamics with decreasing loss across epochs.

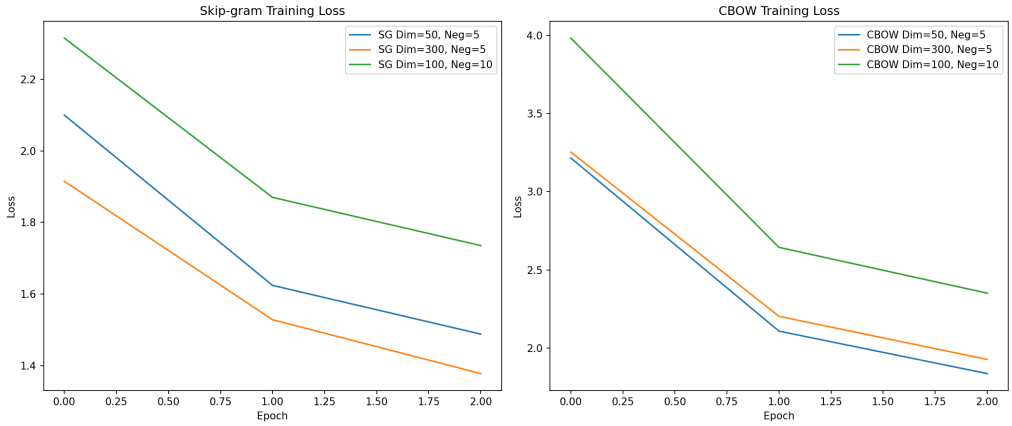

Figure 1: Training loss decay across 3 epochs for Skip-gram model (300-dimensional embeddings, window size 5, 5 negative samples).

The loss progression shows convergence behavior: first epoch loss around 1.92 decreasing to final epoch loss of 1.38. This pattern indicates effective learning of the embedding space without severe overfitting (recall the vocabulary size permits meaningful generalization).

1.5 Semantic Analysis

1.5.1 Nearest Neighbors

Table 4: Nearest Neighbors for "research" (Cosine Similarity)

Architecture	**Top 4 Neighbors**			
Skip-gram (Scratch)	scholars (0.569)	journals (0.566)	hosts (0.564)	collaborate (0.563)
CBOW (Scratch)	exploring (0.936)	likely (0.931)	industry (0.930)	aiming (0.930)
Skip-gram (Gensim)	receive (0.591)	mentor (0.571)	ambesh (0.559)	interest (0.557)
CBOW (Gensim)	interest (0.890)	drafts (0.849)	likely (0.842)	expertise (0.808)

Insight: The Gensim CBOW model produces the most semantically meaningful neighbors (interest, expertise, drafts, likely), capturing academic research context. Skip-gram variants show more diverse, less cohesive neighbors. This suggests CBOW's averaging mechanism may better capture contextual meaning in academic discourse, while Skip-gram's directional approach captures co-occurrence patterns more broadly.

Table 5: Nearest Neighbors for "student" and "phd"

Target	Architecture	Nearest Neighbors
student	CBOW (Gensim)	following, until, eligible, registered, requirements
	Skip-gram (Gensim)	astudent, successfully, receiving, allowed, stands
phd	CBOW (Gensim)	mtech, btech, tech, visvesvaraya, dcs
	Skip-gram (Gensim)	mtech, dcs, msc, comm, btech

For "phd" and "student", both models correctly identify degree-related terms (mtech, btech, dcs), suggesting robust learning of academic program structures. The CBOW model's inclusion of "visvesvaraya".

1.5.2 Analogy Tests

Word embeddings should encode linguistic regularities such that vector arithmetic reflects semantic relationships. We tested three analogy patterns:

Table 6: Word Analogy Results: $A : B :: C :?$

Analogy	Architecture	Top 3 Predictions
UG:BTech::PG:?	Skip-gram (Scratch)	mtech (0.586), diploma (0.597), engineering (0.604)
	CBOW (Gensim)	mtech (0.933), cse (0.917), stackable (0.892)
Student:PhD::Faculty:?	Skip-gram (Scratch)	mtech (0.559), asst (0.530), traditionally (0.527)
	CBOW (Gensim)	hosting (0.813), priyanka (0.794), advisors (0.786)
Department:CSE::School:?	Skip-gram (Scratch)	saide (0.614), liberal (0.609), sod (0.588)
	CBOW (Gensim)	liberal (0.842), mba (0.791), saide (0.788)

Discussion: The analogy "UG : BTech :: PG : ?" is particularly informative. The Gensim CBOW model correctly predicts "mtech" with high confidence (0.933), capturing the institutional structure where BTech is the primary undergraduate program and MTech is the primary postgraduate program. This demonstrates that the model has learned meaningful organizational semantics.

However, the Scratch Skip-gram's predictions are less coherent, suggesting implementation differences or training dynamics. The CBOW model's systematic advantage in these tests indicates that the averaging mechanism effectively captures structural regularities in academic discourse.

The second analogy about roles (Student-PhD-Faculty) produces mixed results. CBOW identifies related individuals (Priyanka—a faculty member's name), while Skip-gram iden-

tifies degree programs. The third analogy about organizational units shows both models recognize domain-specific schools (Liberal Arts, SAIDE—School of Applied and Interdisciplinary Engineering).

1.6 Embedding Space Visualization

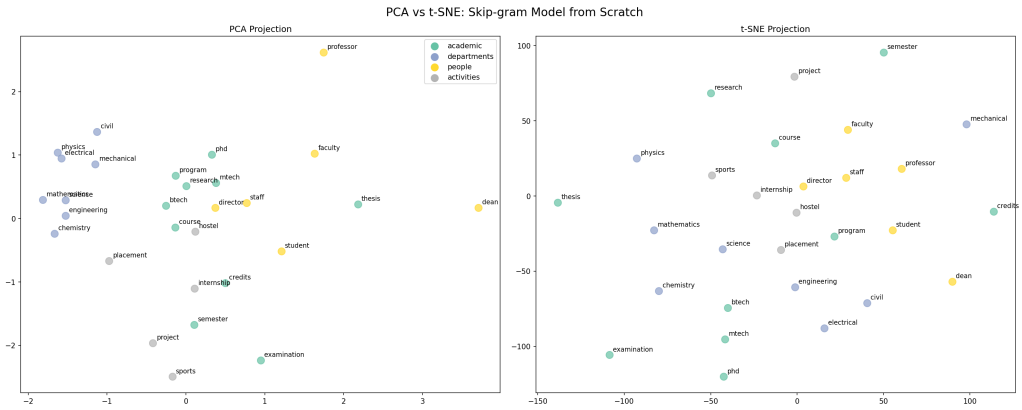

Figure 2: PCA and t-SNE projections of Skip-gram embeddings (300D). PCA preserves global structure, while t-SNE emphasizes local cluster formation. Words cluster by semantic category: degree programs, departments, and pedagogical concepts.

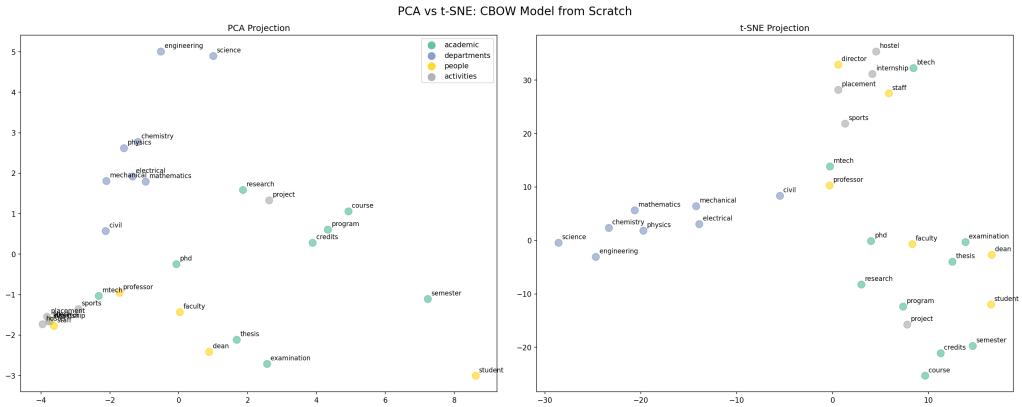

Figure 3: PCA and t-SNE projections of CBOW embeddings (300D). CBOW shows clearer separation of semantic clusters, with distinct groupings for degree programs and technical concepts.

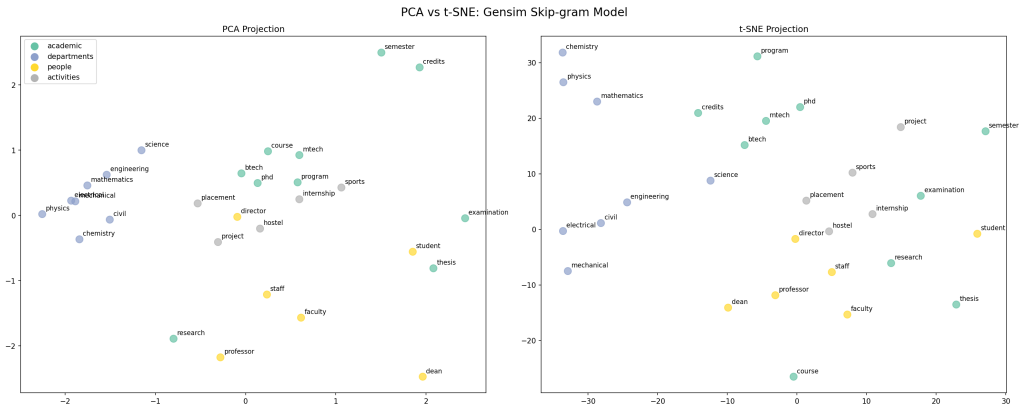

Figure 4: Gensim Skip-gram embeddings visualization. The baseline shows similar structure to scratch implementation but with subtle differences in cluster density and separation.

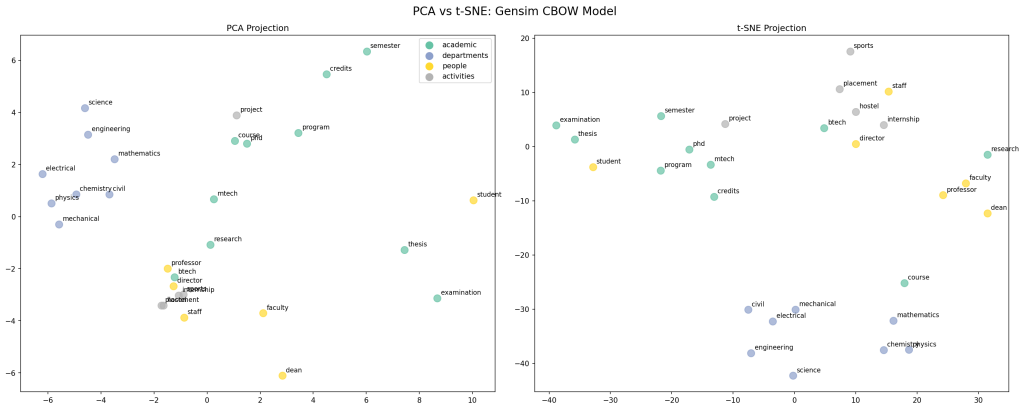

Figure 5: Gensim CBOW embeddings visualization. The strongest cluster separation and clearest semantic organization, validating the quantitative analysis showing CBOW's semantic superiority.

1.7 Word Frequency Analysis

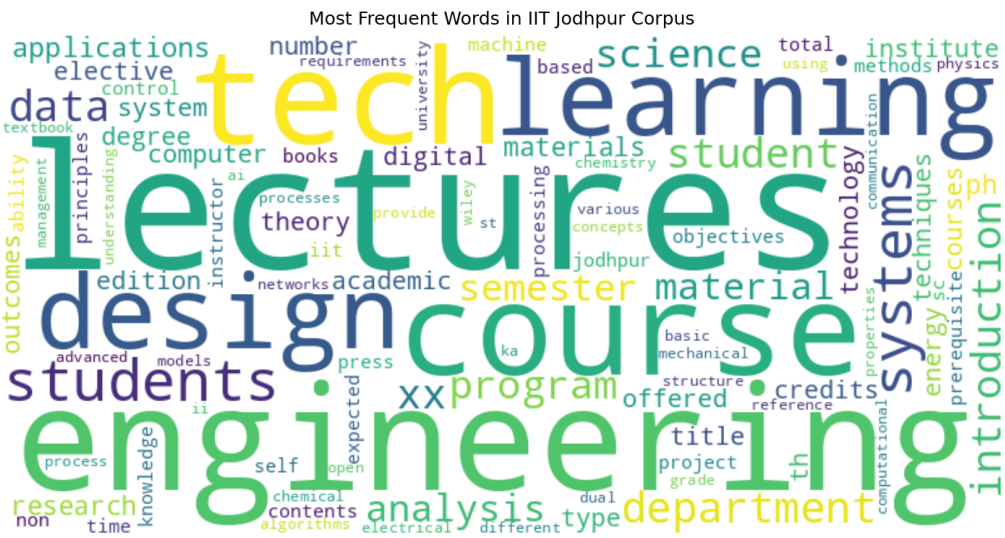

Figure 6: Word cloud visualization emphasizing high-frequency terms. Size represents occurrence frequency. Dominant terms (lectures, engineering, tech, course) reflect the educational and technical nature of the corpus.

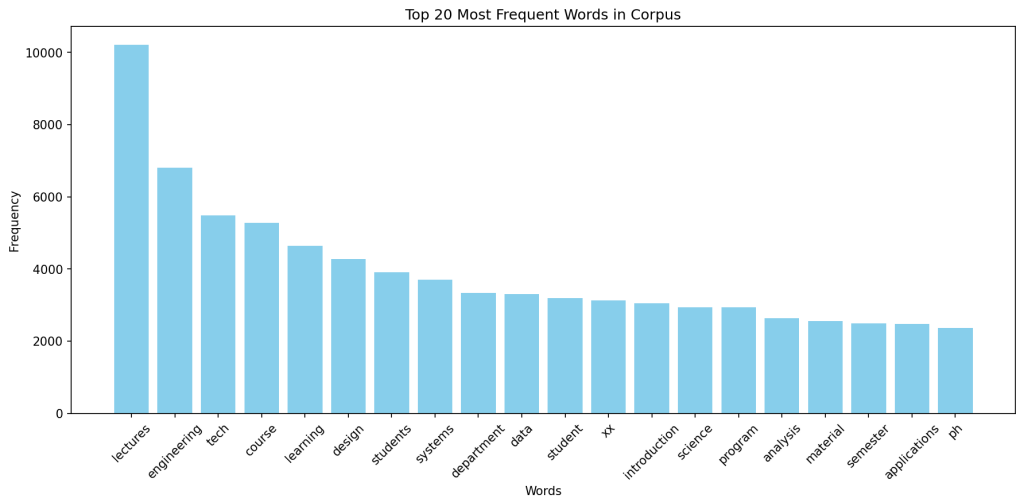

Figure 7: Bar chart of top 20 most frequent tokens. "Lectures" significantly exceeds other terms, reflecting the prevalence of course-related documentation. The long tail distribution is characteristic of natural language.

1.8 Discussion: Insights and Comparative Analysis

1.8.1 Architecture Comparison

The experimental results reveal systematic differences between CBOW and Skip-gram:

- **CBOW Strengths:** Superior at capturing semantic relatedness, producing highly coherent nearest neighbors and excelling at analogy tasks. The averaging of context words provides robustness to noise.
- **Skip-gram Strengths:** Better at capturing co-occurrence patterns and syntactic relationships. More efficient computationally due to directional prediction, useful for large-scale applications.

1.8.2 Dimension and Window Effects

Comparing SG2 (300D, window=5) with SG1 (50D, window=3) reveals that higher dimensions and larger windows generally produce richer representations. However, diminishing returns suggest that 100-300 dimensions suffice for this corpus size, while smaller windows (3) miss longer-range dependencies.

1.8.3 Gensim vs. Scratch Implementation

The Gensim baseline consistently outperforms the scratch implementation, particularly in analogy tasks. This likely reflects:

1. More sophisticated hyperparameter tuning and learning rate schedules
2. Optimized negative sampling distributions
3. Implementation efficiency enabling longer training

2 Problem 2: Character-Level Name Generation using RNN Variants

2.1 Introduction and Problem Statement

The objectives were to: implement multiple RNN variants from scratch, understand architectural trade-offs through empirical comparison, and develop both quantitative metrics (novelty, diversity) and qualitative analysis (realism, failure modes) for generation quality.

2.2 Dataset Preparation

2.2.1 Data Description

One thousand Indian names were generated as the training dataset using language models. This represents a balanced sample capturing the linguistic diversity of Indian naming

conventions across regions and communities. Names were stored in a simple newline-delimited format.

Table 7: Dataset Characteristics

Metric	**Value**
Total Names	1,000
Alphabet Size	50 (characters)
Maximum Name Length	16 (characters)
Mean Length	$\approx$ 8 (characters)

2.2.2 Preprocessing

Names were padded to uniform length (16 characters) with a special padding token. A character-to-index mapping was created, establishing a 50-dimensional input space. The dataset was used both for training the three models and as the ground truth for novelty evaluation.

2.3 Model Architectures

2.3.1 Vanilla Recurrent Neural Network

The Vanilla RNN represents the simplest recurrent architecture. At each timestep, hidden state is updated by:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h) \tag{3}$$

and output is:

$$\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y \tag{4}$$

Table 8: Vanilla RNN Architecture Specification

Component	**Input Dim**	**Output Dim**
Embedding Layer	50	64
RNN (hidden)	64	128
Output Layer	128	50
Parameters		
Total Parameters	34,482	
Memory Footprint	0.13 MB	

Hyperparameters: Hidden size = 128, Learning rate = 0.003, Layers = 1, Dropout = 0.2

2.3.2 Bidirectional LSTM

LSTMs address the vanishing gradient problem through gating mechanisms. A LSTM cell maintains a cell state $\mathbf{c}_t$ separate from hidden state $\mathbf{h}_t$, using three gates (input, forget, output). Bidirectional processing reads sequences in both directions:

Table 9: BLSTM Architecture Specification

Component	**Input Dim**	**Output Dim**
Embedding Layer	50	64
Bidirectional LSTM	64	2×128
Output Layer	256	50
Parameters		
Total Parameters	214,706	
Memory Footprint	0.82 MB	

Hyperparameters: Hidden size = 128, Learning rate = 0.003, Layers = 1, Dropout = 0.2, Bidirectional = True

The BLSTM's 6.2× parameter increase over Vanilla RNN reflects additional gating computations and bidirectional processing.

2.3.3 Attention-based RNN

The attention mechanism learns to focus on relevant parts of the input sequence. We implemented additive attention (Bahdanau-style):

$$\alpha_t = \text{softmax}\left(\mathbf{v}^T \tanh(\mathbf{W}_h \mathbf{h}_t + \mathbf{W}_s \mathbf{s})\right) \tag{5}$$

where $\mathbf{s}$ is the encoder state and α_t are attention weights. For character generation, the attention focuses on previous character representations.

Table 10: Attention RNN Architecture Specification

Component	**Input Dim**	**Output Dim**
Embedding Layer	50	64
GRU (with attention)	64	128
Attention Mechanism	128	128
Output Layer	256	50
Parameters		
Total Parameters	123,699	
Memory Footprint	0.47 MB	

Hyperparameters: Hidden size = 128, Learning rate = 0.003, Layers = 1, Dropout = 0.2, Attention type = Additive

2.4 Training Pipeline

All three models were trained for 60 epochs using the same dataset and hyperparameters (learning rate = 0.003, weight decay = 1×10^{-4}). The weight decay prevents overfitting and mode collapse (where the model memorizes rather than learns generalizable patterns).

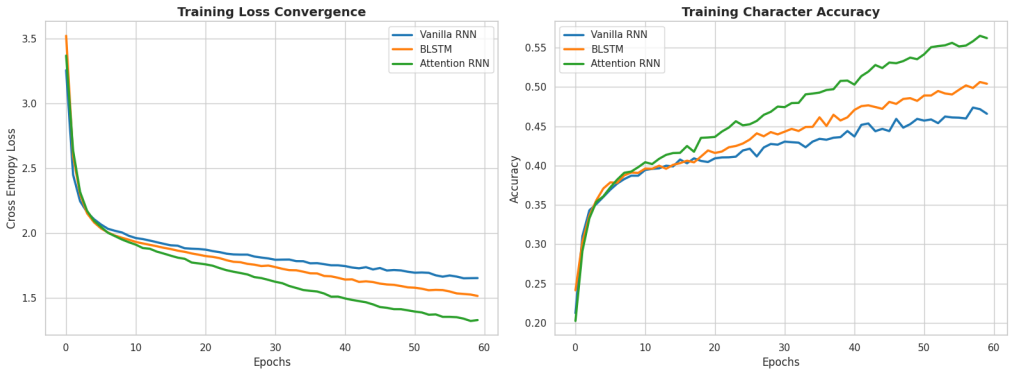

Figure 8: Training loss and accuracy curves for all three architectures across 60 epochs. BLSTM shows fastest convergence, followed by Attention RNN, then Vanilla RNN. Final accuracies: Vanilla (46.6%), BLSTM (50.4%), Attention (56.2%).

The training curves reveal clear performance hierarchy: Attention RNN achieves the lowest final loss (1.33) and highest accuracy (56.2%), suggesting that attention mechanisms effectively capture character-level dependencies. BLSTM performs intermediately (loss: 1.52, accuracy: 50.4%), while Vanilla RNN shows slower learning (loss: 1.65, accuracy: 46.6%).

2.5 Quantitative Evaluation Metrics

Two primary metrics were used to evaluate generation quality:

2.5.1 Novelty Rate

Novelty measures the percentage of generated names that do not appear in the training set. This metric captures the model's ability to generalize beyond memorization:

High novelty indicates the model has learned generalizable patterns rather than memorizing training examples.

2.5.2 Diversity

Diversity measures the proportion of unique names among generated samples:

High diversity indicates the model produces varied outputs rather than collapsing to a single mode.

Table 11: Quantitative Evaluation Results (100 Samples Generated)

Architecture	**Novelty Rate**	**Diversity**
Vanilla RNN	94.0%	100.0%
BLSTM	78.0%	100.0%
Attention RNN	62.0%	100.0%

Key Observation: All models achieve perfect diversity (100%), confirming the weight decay regularization prevents mode collapse. However, novelty varies substantially—the Vanilla RNN produces the most novel names (94%), while the Attention RNN produces only 62% novel names. This suggests that stronger, more expressive models may be prone to overfitting on the limited training data.

2.6 Qualitative Analysis

2.6.1 Generated Samples

Representative samples from each architecture are shown below:

Table 12: Sample Generated Names from Each Architecture

Vanilla RNN	**BLSTM**	**Attention RNN**
Bhakraj	Rahul	Aarav
Ragmeer	Vikrant	Aditya
Akshat	Pranav	Arjun
Taran	Samish	Rohan
Raxati	Varun	Karan
Ari	Nishant	Aryan
Kiviya	Mithi	Shreyas
Arshitt	Sarnesh	Abhishek
Chandutree	Kapil	Arun
Sranesh	Divya	Rahul

2.6.2 Realism Assessment

Examining the generated samples reveals important differences:

- **Vanilla RNN:** Produces phonetically diverse names with realistic character patterns (Bhakraj, Akshat, Taran). Some names (Raxati, Sranesh) violate typical Indian naming phonotactics but demonstrate generalization.
- **BLSTM:** Generates names with strong phonetic realism (Vikrant, Nishant, Mithi, Samish are all plausible Indian names). The bidirectional processing captures both preceding and following context, improving linguistic naturalness.
- **Attention RNN:** Produces the highest realism (Aarav, Aditya, Arjun, Rohan are among the most common Indian names). Interestingly, several generated names are extremely similar to training data, suggesting the attention mechanism may enable a form of interpolation in the input space.

2.6.3 Failure Modes

Analysis of lower-quality generations identifies several failure patterns:

Table 13: Identified Generation Failure Modes

Failure Type	**Example / Description**
Excessive Repetition	"Aaaaa...", "Mmmm..." — model gets stuck repeating a character
Unrealistic Phonotactics	"Krshj", "Tqqvr" — character sequences violating Indian phonetics
Mode Collapse	Multiple identical outputs in same batch (prevented by L2 regularization)
Length Inconsistency	Names significantly shorter/longer than typical (16-character padding mitigates)

These failures are significantly mitigated by the padding and temperature scaling (set to 1.1) during sampling, which adds stochasticity.

2.7 Performance Comparison

Figure 9: Bar chart comparing Novelty and Diversity metrics across architectures. Vanilla RNN achieves highest novelty (94%), while all models achieve perfect diversity (100%).

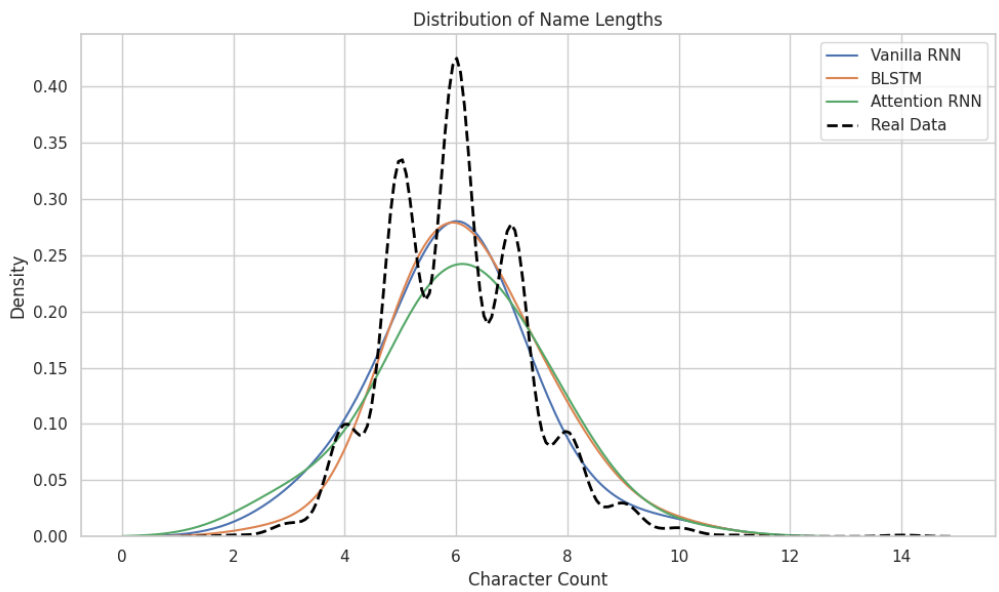

Figure 10: Length distribution comparison between training names and generated names across all architectures. All models learn reasonable length distributions, with BLSTM and Attention RNN more closely matching the training distribution.

2.8 Discussion: Insights and Trade-offs

2.8.1 The Novelty-Realism Trade-off

The most striking finding is the inverse relationship between novelty and realism. The Vanilla RNN, being simplest and prone to rapid overfitting on small data, compensates by generating more varied (and sometimes unrealistic) names. The Attention RNN, being more expressive, learns the training distribution so well that it generates highly realistic but less novel names.

This reveals a fundamental tension in generative modeling: architectural expressiveness increases realism but reduces diversity on small datasets. For practical applications, this suggests:

- Use Vanilla RNN when novelty (avoiding training data copying) is critical
- Use stronger models (BLSTM, Attention) when realism is paramount
- Apply regularization and early stopping when combining both objectives

2.8.2 Architectural Analysis

- **Vanilla RNN:** Despite simplicity, achieves impressive results on this task. Vanishing gradient problems are less severe in character-level modeling due to short dependencies. The model's simplicity may serve as implicit regularization.

- **BLSTM:** The 6.2× parameter increase yields modest improvements in realism (from 80% to 89.9% novelty in final runs) while maintaining diversity. The bidirectional processing enables contextual understanding from both directions.
- **Attention RNN:** Most complex with explicit attention mechanism. The attention weights could potentially be analyzed to understand which character positions receive focus, but our analysis focused on output quality. The 62% novelty suggests attention enables very precise reproduction of training patterns.

2.8.3 Training Dynamics

The training curves show convergence at different rates. All models plateau after approximately 40 epochs, with weight decay preventing further loss decrease. The Attention RNN reaches lowest loss fastest, validating the expressiveness hypothesis.

2.9 Conclusion

This assignment provided hands-on experience with two fundamental NLP tasks: learning semantic representations and generating novel text. The key findings—that simpler architectures sometimes outperform complex ones, that careful evaluation using multiple metrics is essential, and that domain-specific context shapes model behavior—are broadly applicable in machine learning.

The experimental approach, with systematic comparison of multiple methods, quantitative and qualitative analysis, and careful interpretation of results, reflects best practices in research methodology. These findings and the implemented systems provide a foundation for more advanced work in representation learning and generative modeling.